

Abstração em UML e Java

Introdução

A abstração é um dos conceitos fundamentais na programação orientada a objetos, permitindo simplificar e organizar sistemas complexos. Nesta aula, abordaremos a aplicação desse conceito em UML e Java, explorando como a abstração nos ajuda a focar nos aspectos essenciais de um sistema, omitindo detalhes irrelevantes. Em UML, utilizamos diagramas para representar visualmente classes, suas propriedades e comportamentos. Já em Java, as classes implementam diretamente essa abstração, facilitando o desenvolvimento de software. Além disso, veremos como o Astah, uma ferramenta de modelagem UML, permite gerar automaticamente código Java a partir de diagramas, integrando design e implementação de forma eficiente.

Abstração em UML

A abstração em UML é uma ferramenta poderosa para lidar com a complexidade de sistemas, permitindo focar apenas nos aspectos mais essenciais. No processo de desenvolvimento de software, é comum lidarmos com objetos que possuem uma infinidade de atributos e comportamentos. No entanto, para projetar sistemas de forma eficiente, devemos selecionar e modelar apenas os atributos e comportamentos mais importantes para o contexto em questão. Um exemplo prático seria o uso de um carro em diferentes cenários: em um estacionamento, o atributo mais relevante pode ser a placa; para um serviço de aplicativo de transporte, a cor ou o modelo podem ser mais importantes; e em uma oficina mecânica, detalhes sobre avarias seriam essenciais. Assim, o conceito de abstração nos ajuda a omitir detalhes que não são importantes para o contexto atual.

Na UML, a abstração é representada por meio de diagramas, como diagramas de classe, que ajudam a simplificar e visualizar a arquitetura do sistema. Essa abordagem visual facilita a comunicação entre

desenvolvedores, analistas e outros stakeholders envolvidos no projeto. Por exemplo, o diagrama de classes é uma forma de abstração em que as classes representam objetos do mundo real de forma simplificada. Além disso, diagramas de caso de uso e diagramas de sequência também utilizam o conceito de abstração, permitindo que diferentes aspectos do sistema sejam modelados de maneira clara e compreensível.

A abstração em UML também desempenha um papel crucial no planejamento da arquitetura de sistemas, sem a necessidade de mergulhar nos detalhes da implementação. Isso é especialmente útil para arquitetos de software que precisam ter uma visão global do sistema antes que os engenheiros de software iniciem o processo de codificação. O uso de conceitos como generalização, especialização e herança permite criar hierarquias e categorias de classes, facilitando a organização e a estruturação do sistema. Por fim, a abstração em UML promove um pensamento de alto nível, permitindo que o design do software seja eficiente e bem estruturado desde o início.

Criação de classes no Astah

Na criação de classes usando o Astah, começamos selecionando o “Diagrama de Classe” no menu principal da ferramenta. Esse diagrama é essencial na modelagem de sistemas em UML, pois permite visualizar a estrutura de classes e seus atributos. Para exemplificar, usamos a classe “Conta Bancária.” Inserir uma nova classe é simples: basta dar um duplo clique na área de trabalho do Astah e nomear a classe. Feito isso, o próximo passo é adicionar atributos, como número da conta, titular, saldo e limite. Esses atributos são definidos utilizando boas práticas de nomenclatura, com letras minúsculas iniciais, e atribuímos tipos adequados, como “string” para o número da conta e o titular, e “double” para o saldo e o limite.

Após definir os atributos, inserimos os métodos, ou operações, da classe. Métodos como getNum, getTitular, getSaldo e getLimit são usados para obter os valores dos atributos, enquanto os métodos setters, como setTitular e setLimit, permitem alterar esses valores. É importante que os métodos get retornem os tipos corretos de dados. Por exemplo, o método getSaldo retorna um valor “double,” já que lida com valores numéricos. Para os setters,

utilizamos “void” como retorno, pois esses métodos apenas alteram os valores, sem retornar dados. A exceção é o método sacar, que retorna um valor booleano, indicando o sucesso ou falha da operação.

A visibilidade dos atributos e métodos é um aspecto crucial da modelagem em UML. No Astah, usamos símbolos como o traço (-) para indicar que um atributo é privado e o sinal de mais (+) para indicar que é público. Seguindo as melhores práticas de encapsulamento, configuramos os atributos como privados e os métodos como públicos. Além disso, o Astah nos permite marcar atributos como estáticos, quando necessário, e configurar relações de multiplicidade entre classes.

Abstração em Java

Abstração em Java é um conceito fundamental da programação orientada a objetos, onde classes servem como modelos para a criação de objetos. Uma classe define o estado de um objeto através de seus atributos e seu comportamento através dos métodos. Em Java, a classe também implementa diretamente o conceito de abstração, facilitando a criação de objetos que possuem apenas os atributos e métodos relevantes para o contexto do sistema em desenvolvimento.

Por exemplo, em uma classe “Funcionário”, que pode ser definida tanto em UML quanto em Java, temos os atributos nome, cargo e salário, onde os dois primeiros são públicos e o terceiro, privado. Além disso, métodos como calcular bônus (público) e aumentar salário (privado) definem o comportamento da classe. A classe em Java começa com a declaração de visibilidade (“public”) e o nome da classe, seguida pela declaração dos atributos e métodos, respeitando as convenções de nomenclatura e visibilidade.

Ao implementar a classe em um ambiente de desenvolvimento como o Android Studio, o processo se torna mais eficiente graças às funcionalidades automáticas, como o autocompletar, que sugere métodos get e set para os atributos, e o fechamento automático de chaves. Isso ajuda a evitar erros de sintaxe e torna o código mais organizado. A abstração em Java permite que o

desenvolvedor se concentre nos aspectos essenciais do sistema, deixando de lado detalhes irrelevantes, o que é fundamental para a criação de softwares bem-estruturados.

Geração de código Java no Astah

A geração de código Java no Astah é uma funcionalidade poderosa que permite criar código-fonte diretamente a partir de diagramas UML. Para começar, é importante configurar corretamente a linguagem de programação, que neste caso é Java. Isso é feito no menu “Project,” onde definimos a linguagem utilizada no projeto. Depois disso, podemos selecionar uma classe ou até mesmo todo o diagrama de classes para gerar automaticamente o código Java.

Ao acessar o menu “Tools” no Astah, podemos exportar a classe ou o diagrama para código Java. No exemplo da classe “Funcionário”, a ferramenta gera automaticamente uma classe pública, com atributos como nome, cargo e salário, respeitando as configurações de visibilidade e tipos definidas no diagrama. Os métodos, como calcular bônus e calcular salário, também são gerados com a nomenclatura correta. Se algum método não tiver um tipo de retorno especificado no diagrama, o Astah insere um retorno padrão, como “zero.”

Essa integração entre UML e Java é extremamente útil, pois permite que o código-fonte e os diagramas se mantenham sincronizados. Além disso, é possível importar código-fonte para gerar diagramas automaticamente, o que facilita o trabalho de documentação e análise de sistemas complexos. Com essa funcionalidade, a modelagem UML se torna ainda mais prática, integrando diretamente a fase de design com a implementação em Java.

Essa é a última parte da nossa aula sobre “Abstração em UML e Java”. Nos tópicos anteriores, abordamos a criação de classes no Astah e a aplicação dos conceitos de abstração em Java. Agora, você tem uma visão completa de como utilizar o Astah para modelar sistemas e gerar código de maneira eficiente.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Desvendando a Abstração na Programação Orientada a Objetos | Tudo em detalhes...”, que está disponível no canal Giaretta no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. Fundamentos da programação de computadores. 3.ed. Pearson: 2012.

BRAGA, P. H. Teste de software. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). Arquitetura de software. Pearson: 2017.

GALLOTTI, G. M. A. Qualidade de software. Pearson: 2015.

MEDEIROS, E. Desenvolvendo software com UML 2.0 definitivo. Pearson: 2004.

MORAIS, I. S. de. (Org.). Engenharia de software. Pearson: 2017.

PFLEEGER, S. L. Engenharia de software: teoria e prática. 2.ed. Pearson: 2003.

SOMMERVILLE, I. Engenharia de software. 10.ed. Pearson: 2019.